Towards Recursive Self Home Improvement

Lifting a weak open model at dishwasher loading with a self-rewriting team

RSHI Team Claude (Anthropic)*

Abstract

How best to load a dishwasher is a combinatorial, physical problem with an unusually honest reward: a simulated wash decides which items come out clean [8]. We use it as a testbed for self-improving multi-agent systems built only from open, inexpensive models. A deliberately weak loader (Nemotron-3.5-Lightning-30B-A3B, thinking off) is surrounded by a planner, a vision critic, a coach and a librarian, all open Nemotron models on Crusoe serverless inference. Between generations the team rewrites a typed genome of its own prompts, playbook, tool policy and structure, and a deterministic gatekeeper keeps an edit only if it beats the incumbent on paired, fresh tasks. On a held-out exam whose levels contain item kinds and soils never seen in training, the evolved team scores ?? against ?? for the bare loader (?? pp). We report what the loop changed, what it cost, and why the evidence is still thin.

1 Introduction

Recursive self-improvement is usually discussed for systems that rewrite their own weights. A humbler, testable version keeps the model fixed and lets the system rewrite the *scaffold* around it: prompts, memory, tools and the division of labour [20, 65, 67]. Two things make such loops hard to trust. The reward is often an LLM judge, which can be flattered and drifts [70], and a loop that optimises a proxy finds the proxy's seams [27, 33, 52]. And improvements are usually measured on the distribution they were selected on.

We take the problem of loading a dishwasher, using DishwasherBench, Composo's open (MIT) benchmark [8]. It models one real machine (Bosch SMV4HAX40G): items are placed with deterministic rigid-body physics (Jolt) through 13 MCP tools, then a GPU wash simulation (WebGPU via Dawn) of the Eco 50 programme with one detergent capsule, calibrated on washes of the real machine (parameter set `anchored-v8.owner-capsule`), decides which items are clean against a threshold derived from the IEC 60436 scale [6, 12, 21]. An item scores its points only if it comes out clean. The reward is *objective* (points over points available), *deterministic* (a load always settles and washes the same way, and every episode is verifiable by replay) and *unfakeable*: a bowl mouth-up under a dead spot of the spray arm stays dirty however eloquently the agent describes it. It is also hard: not everything fits, so the player chooses what to load as well as how; jets reach only part of the load [43, 44]; and choosing a subset under capacity is a knapsack over strongly NP-hard 3D packing [24, 34]. Household benchmarks score symbolic goal predicates [45, 49, 51, 55], and prior dishwasher-loading work scores grasp success or agreement with a human preference [22, 23, 30, 56]; to our knowledge none scores whether the items come out clean, the gap DishwasherBench fills.

The benchmark, simulator, calibration, levels and harness are Composo's work [8]. Ours is the self-improving team (*crew*), its memory, Gatekeeper, exam protocol and visualiser, plus cosmetic water, watch and turntable changes to the benchmark's web client that touch neither simulation nor scoring. We claim that a team of cheap open models can lift a weak loader by rewriting the system around it, keeping only changes that measurably help, and that the lift survives on unseen tasks. Contributions:

- A team whose rewritable state is a typed, versioned *genome*, edited only through six typed operations, with the judging roles (memory accounting, acceptance) in deterministic code: the team can change how it plays, never how it is graded.
- A paired acceptance rule over shared seeds in a deterministic simulator, a three-layer memory with shrunken Bayesian confidence, and a held-out exam on task families with new kinds and soils.
- An end-to-end run on open Nemotron models with lineage, costs and limits reported.

2 Related Work

Scaffold self-improvement. STOP recursively improves the improver around a fixed LM [65]; ADAS has a meta-agent program new agents [20]; the Darwin Gödel Machine keeps self-edits that validate, in an open-ended archive [67]; Gödel Agent and SICA rewrite their own logic or code [47, 63]. AFlow searches workflows with MCTS and lets cheap models match expensive ones [66]; GPTSwarm optimises prompts and agent connectivity [71]; FunSearch and AlphaEvolve gate LLM proposals with automatic evaluators [36, 48]. Our Coach is a typed, constrained meta-agent in this line; we differ in the physical reward, the paired gate and held-out families.

Prompt and program optimisation. OPRO optimises from a scored history [62]; Promptbreeder and EvoPrompt evolve prompts by LLM mutation [13, 16]; DSPy and MIPRO tune multi-stage LM programs against a metric [25, 40]; TextGrad and Trace propagate textual feedback [7, 64]; GEPA's reflection on traces is closest to our Critic→Coach loop [1]. We widen the edit space to tools and team structure.

Memory and multi-agent systems. Reflexion stores verbal reflections [50]; ExpeL and CLIN distil and transfer cross-task insights [32, 68]; Voyager and AWM grow verified skill or workflow libraries [59, 60]; Generative Agents and MemGPT structure retrieval under a context budget [41, 42]. Unlike self-refinement [31], our lessons are counted against physics. Role-based teams [19, 28, 46, 61] and debate [10] follow fixed processes; ours edits its own structure. Voting and repeated sampling with a verifier [4, 29, 53] trade compute for parameters;

*Built at the Crusoe × Cerebral Valley Recursive Agents Hackathon, 17 October 2026. Code: `https://github.com/elevenarms/RSHI`.

Role	Model (open, on Crusoe)	Job
Loader	Nemotron-3.5-Lightning-30B-A3B	plays, 13 tools
Planner	Nemotron-3.5-Super-VL	per-item plan
Critic	Nemotron-3-Nano-Omni-Reas.-30B-A3B	post-mortem
Coach	Nemotron-3-Super-120B-A12B	genome edits
Librarian	Nemotron-3-Nano-30B-A3B + code	memory, brief
Gatekeeper	deterministic statistics	accept/reject

Table 1: Roles and models [37–39]; loader thinking off.

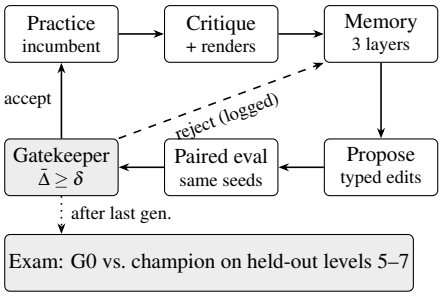


Figure 1: One generation of the loop. Rejections (title, hypothesis, per-task deltas) go to procedural memory and are shown to the Coach; after the last generation the champion sits the exam.

our simulator is such a verifier, but far too slow to call per action.

Search over loads. Loading is constrained packing with a physical, black-box objective [58, 69]; genetic algorithms [15, 18], CMA-ES over poses [17], quality-diversity [35], Bayesian optimisation [54] and MCTS over placement sequences [5, 26] are natural per-instance optimisers. We do not search loads directly: every evaluation costs a wash, and the product is *one load*, not a better loader, so nothing transfers to a new set of items. We optimise the system, and test whether its improvements carry to tasks they were not selected on. Search is a natural future tool for the team.

3 Method

The team (Table 1, Fig. 1). The Loader plays through the standard DishwasherBench harness tools [8], text only. Optionally a Planner turns the level card and a memory brief into a per-item plan (rack, stance, what to leave out). After each practice episode the Critic gets the item-by-item outcome (rack, stance, clean, residue regions, retained water, movement), the plan, the playbook, the loader's last words and two renders of the final load, and writes up to five tagged lessons with a polarity. The Coach proposes edits; the Librarian condenses memory into a brief.

Genome and typed edits. A genome is an immutable record of everything the team may rewrite: three prompts (Planner, loader preamble, Critic); a playbook of at most 25 tagged rules with evidence links to lessons; team switches (`usePlanner`, `useMemoryBrief`, `preflightCheck`, which refuses `start_cycle` before a read `check_load`, and the brief's lesson count); and loader settings (reasoning effort, images, denied tools, turn limit). The Coach can change it only by `add_rule`, `edit_rule`, `remove_rule`, `set_prompt`, `set_team` and `set_loader`. A proposal carries a title and a falsifiable hypothesis, should change one coherent thing so gains can be attributed, and is shown past rejections. Invalid edits (ranges, sizes, tools that may never be denied) are dropped; a child's id hashes its content with its parent, so the lineage is diffable. The baseline G0 is the bare loader: no planner, no brief, no preflight, an empty playbook, 120 turns.

Three-layer memory. The *episodic* layer keeps every training episode item by item; the *semantic* layer keeps lessons (from the Critic, and observed from placement statistics), deduplicated on merge, with support s and contradiction c counts; the *procedural* layer keeps every genome and decision. Confidence is a Beta(1,1) posterior mean shrunk towards ignorance while evidence is thin:

$$p = \tfrac{s+1}{n+2}, \quad \kappa = \tfrac{1}{2} + \left(p - \tfrac{1}{2}\right)\tfrac{n}{n+K}, \quad n = s+c,\ K = 3, \qquad (1)$$

so one supporting episode gives 0.54, five of five 0.72, twenty of twenty 0.90. A level's brief lists smoothed clean rates $(k+1)/(m+2)$ of its kinds per rack and stance, then lessons with $\kappa \geq 0.5$ ranked by $\kappa \cdot \text{relevance} \cdot \log(1+n)$. Exam episodes are stored apart and never read back.

Gatekeeper. Candidate C and incumbent I play the same fresh validation drills with the same seeds. Let P be the drills both played validly and x_t the share on drill t; then

$$\bar{\Delta} = \tfrac{1}{|P|}\textstyle\sum_{t \in P}\left(x_t^C - x_t^I\right), \qquad \text{accept} \iff \bar{\Delta} \geq \delta, \qquad (2)$$

and the decision is void if $|P| < 3$. Agent failures (turn or token limit, nothing loaded) score what they scored; *invalid* episodes (provider or harness errors) are replayed on the same task and seed up to twice, and drills still unpaired are dropped for both genomes.[1] Since loading and washing are deterministic, pairing cancels drill difficulty, leaving the genome's effect plus sampling noise. The accepted candidate with the largest $\bar{\Delta}$ becomes the incumbent; its validation episodes are the next practice set.

Exam: what "held-out" means. Training and validation drills come from DishwasherBench course levels 1–4 (tutorial, lunch for four, weeknight dinner, breakfast), the exam from levels 5–7 (mixed family, baking day, dinner party) [8]. These are new task families, not just new seeds: they hold kinds and soils training never showed (baby bottle and teat, spouted beakers, lunch tubs, baking tray, tin, platter, board; burnt sugar, overnight soils). G0 and the champion play identical exam drills; nothing is learned from them.

4 Experimental Setup

All models run on Crusoe Managed (serverless) Inference via an OpenAI-compatible API, with a client-side limiter (600 requests per minute per model) and byte-stable prompts for prefix caching (83–90% of input tokens hit the cache in smoke runs). The loader is the weakest Nemotron that still plays: 41–70% in smokes, where Nemotron-3-Nano-Omni-Reasoning saturated at 100% and Nano-30B or Omni with thinking off looped on refused calls. A drill is 12 items of a DishwasherBench level

[1] Our first version counted provider errors as agent failures. In generation 2 the fast, non-thinking loader at 8-way concurrency hit Crusoe's per-model request cap (HTTP 429), and both decisions were voided. On cheap shared endpoints rate limits are a real systems constraint, and an acceptance rule must not confuse infrastructure noise with policy quality.

Exam drill	Source	Avail.	G0	Champ.	Δ
Family #0	mixed-family	59	??	??	??
Baking #1	baking-day	71	??	??	??
Party #2	dinner-party	77	??	??	??
Family #3	mixed-family	71	??	??	??
Total points		278	??	??	??
Mean share			??	??	??

Table 2: Held-out exam: points per 12-item drill for G0 and the champion **G?-??????**, same drills and seeds, fast tier.

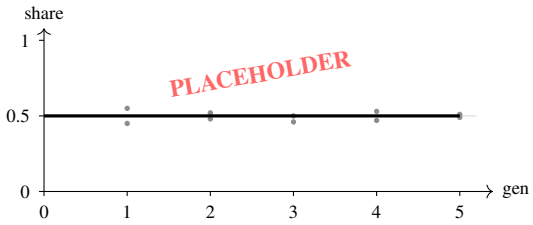


Figure 2: Incumbent mean validation share (steps) and candidates (dots) per generation. Seeds are fresh each generation, so only within-generation differences are comparable.

drawn by seed, cutlery in proportion. We ran 5 generations: 4 baseline practice drills, 2 proposals and 4 paired fresh validation drills per generation, $\delta = 0.03$, then an exam on 4 held-out drills (Table 2). Washes use the fast tier (4 mm blocks rather than 2 mm), whose verdicts agree with the official tier's on 91% of the standard load and 97% of pieces on the course levels [8].

The bare loader often wanders until the turn or token limit (about 115–120 turns, 3M input tokens), sometimes loading nothing (0 of 87 points on one weeknight drill) or few items, yet what it loads mostly comes out clean. It fails at planning and commitment, not physics, which suggests where a team can help.

5 Results

Exam. The champion (**G?-??????**, generation **??**) scores a mean share of **??** against **??** for G0 (**??** pp), ahead on **??** drills (Table 2). It loads **??** items against **??**, with clean-of-loaded rates of **??** and **??**; failed episodes **??** vs. **??**. Read the gain through share = (loaded fraction) × (clean rate): the baseline's failure mode predicts that most of it is the first factor.

Evolution. The Gatekeeper accepted **??** of **??** candidates over 5 generations (Fig. 2); memory ended with **??** lessons from **??** episodes. Generation 1 shows the pattern. "Enable a Planner to guide loading", a structural change, was *accepted* at $\bar{\Delta} = +0.135$ (2 wins, 1 loss, 1 tie); "Add a bowl-mouth-down rule", a playbook rule grounded in a Critic lesson, was *rejected* at -0.095 (1 win, 2 losses). Structure helped the weak loader; a plausible domain rule hurt it, perhaps by adding a constraint it then spent turns satisfying. Accepted edits:

- G1: "Enable a Planner to guide loading" (team structure), $\bar{\Delta} = +0.135$.
- **G?: "title" (op . . .), $\bar{\Delta} = +?.??$.**

Cost. The run took **??** and **??** input tokens (**??** cached) plus **??** output tokens, an inference cost of **??**; caching matters because the loader resends a growing, byte-stable history every turn.

6 Discussion and Limitations

The design point we would defend is the division of authority: the models may propose anything the edit grammar allows, but physics scores it and code decides, which is what makes a loop of cheap, fallible models safe to leave running and what LLM-judged loops lack [2, 70]. The open question is credit assignment: does the lift come from structure (a planner that commits the loader to a list) or from content (rules and lessons about what washes clean)? The baseline's failure suggests structure; transfer to unseen kinds would need some content.

Limitations. *One machine*: one dishwasher, programme and detergent; the exam tests transfer to new items, soils and loads, not new physics. *Small n*: four validation and four exam drills, one exam pass; no confidence interval worth the name, and a paired permutation or sign test [9] on four drills has little power. *Stochastic sampling*: the simulator is deterministic but the models are not, so pairing removes task variance, not policy variance. *Fast tier*: a few percent of verdicts differ from the official tier. *Multiple comparisons*: 2 candidates per generation are each tested at a fixed margin and the best is kept, so validation gains are biased upwards, and repeated accept/reject on one benchmark overfits it [3, 11, 14]; the held-out exam, not the validation curve, is the evidence. *Margin*: $\delta = 0.03$ over four drills is one to two average items (about 70 points over 12 items) on a single drill, below the plausible run-to-run noise of a sampling loader; a sequential test [57] or repeated validation would be the principled fix.

7 Conclusion

A team of open, inexpensive models can rewrite the system around a weak loader, keep only what the physics confirms, and carry the improvement to tasks with objects it has never handled. The dishwasher is a small domain, but its reward cannot be argued with, which is exactly what recursive self-(home-)improvement needs. Next: more exam passes, an official-tier exam, and a search tool (GA or MCTS over placements) that the team may decide to use.

Acknowledgements. We thank Composo for releasing DishwasherBench under the MIT licence, Crusoe for the inference, and NVIDIA for the open Nemotron models.

References

[1] L. A. Agrawal, S. Tan, D. Soylu, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning, 2025. arXiv preprint arXiv:2507.19457.
[2] D. Amodei, C. Olah, J. Steinhardt, et al. Concrete problems in AI safety, 2016. arXiv preprint arXiv:1606.06565.
[3] Y. Benjamini and Y. Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995.
[4] B. Brown, J. Juravsky, R. Ehrlich, et al. Large language monkeys: Scaling inference compute with repeated sampling, 2024. arXiv preprint arXiv:2407.21787.
[5] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.
[6] CENELEC. EN 60436:2020 electric dishwashers for household use – methods for measuring the performance (incorporating IEC 60436:2015+a1:2020). European standard; successor of EN 50242, 2020.

[7] C.-A. Cheng, A. Nie, and A. Swaminathan. Trace is the next AutoDiff: Generative optimization with rich feedback, execution traces, and LLMs, 2024. arXiv preprint arXiv:2406.16218.

[8] Composo. DishwasherBench: A dishwasher-loading game on a physical simulation of one real machine. Software, version 0.1.0. `https://github.com/composo-ai/DishwasherBench`, Oct. 2026. Released 2026-10-12. MIT License. `https://composo.ai`.

[9] R. Dror, G. Baumer, S. Shlomov, and R. Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, 2018.

[10] Y. Du, S. Li, A. Torralba, et al. Improving factuality and reasoning in language models through multiagent debate, 2023. arXiv preprint arXiv:2305.14325.

[11] C. Dwork, V. Feldman, M. Hardt, T. Pitassi, O. Reingold, and A. Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015.

[12] B. Eilts, A. Klingshirn, and D. Boursillon. Dishwasher round robin test 2022/2023: Final report. Technical report, University of Applied Sciences Albstadt-Sigmaringen, Sept. 2023. Sponsored by APPLiA and CENELEC.

[13] C. Fernando, D. Banarse, H. Michalewski, et al. Promptbreeder: Self-referential self-improvement via prompt evolution, 2023. arXiv preprint arXiv:2309.16797.

[14] A. Gelman and E. Loken. The garden of forking paths: Why multiple comparisons can be a problem, even when there is no "fishing expedition" or "p-hacking" and the research hypothesis was posited ahead of time. Working paper, Department of Statistics, Columbia University, 2013.

[15] D. E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading, MA, 1989. ISBN 0201157675.

[16] Q. Guo, R. Wang, J. Guo, et al. EvoPrompt: Connecting LLMs with evolutionary algorithms yields powerful prompt optimizers, 2023. arXiv preprint arXiv:2309.08532.

[17] N. Hansen and A. Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001.

[18] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, MI, 1975. ISBN 0472084607.

[19] S. Hong, M. Zhuge, J. Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework, 2023. arXiv preprint arXiv:2308.00352.

[20] S. Hu, C. Lu, and J. Clune. Automated design of agentic systems, 2024. arXiv preprint arXiv:2408.08435.

[21] International Electrotechnical Commission. IEC 60436:2025 electric dishwashers for household use – methods for measuring the performance. IEC standard; corrected version 2026-04, 2025.

[22] V. Jain, Y. Lin, E. Undersander, Y. Bisk, and A. Rai. Transformers are adaptable task planners. In *Proceedings of the 6th Conference on Robot Learning (CoRL)*, volume 205 of *Proceedings of Machine Learning Research*, pages 1011–1037, 2023.

[23] I. Kapelyukh and E. Johns. My house, my rules: Learning tidying preferences with graph neural networks. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, 2022.

[24] H. Kellerer, U. Pferschy, and D. Pisinger. *Knapsack Problems*. Springer, Berlin, Heidelberg, 2004.

[25] O. Khattab, A. Singhvi, P. Maheshwari, et al. DSPy: Compiling declarative language model calls into self-improving pipelines, 2023. arXiv preprint arXiv:2310.03714.

[26] L. Kocsis and C. Szepesvári. Bandit based Monte-Carlo planning. In *Machine Learning: ECML 2006*, volume 4212 of *Lecture Notes in Computer Science*, pages 282–293. Springer, 2006.

[27] V. Krakovna, J. Uesato, V. Mikulik, et al. Specification gaming: the flip side of AI ingenuity. DeepMind blog, 2020.

[28] G. Li, H. A. A. K. Hammoud, H. Itani, et al. CAMEL: Communicative agents for "mind" exploration of large language model society, 2023. arXiv preprint arXiv:2303.17760.

[29] J. Li, Q. Zhang, Y. Yu, et al. More agents is all you need, 2024. arXiv preprint arXiv:2402.05120.

[30] Y. Lin, A. S. Wang, E. Undersander, and A. Rai. Efficient and interpretable robot manipulation with graph neural networks, 2021. arXiv preprint.

[31] A. Madaan, N. Tandon, P. Gupta, et al. Self-refine: Iterative refinement with self-feedback, 2023. arXiv preprint arXiv:2303.17651.

[32] B. P. Majumder, B. Dalvi Mishra, P. Jansen, et al. CLIN: A continually learning language agent for rapid task adaptation and generalization, 2023. arXiv preprint arXiv:2310.10134.

[33] D. Manheim and S. Garrabrant. Categorizing variants of Goodhart's law, 2018. arXiv preprint arXiv:1803.04585.

[34] S. Martello, D. Pisinger, and D. Vigo. The three-dimensional bin packing problem. *Operations Research*, 48(2):256–267, 2000.

[35] J.-B. Mouret and J. Clune. Illuminating search spaces by mapping elites, 2015.

[36] A. Novikov, N. Vũ, M. Eisenberger, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. arXiv preprint arXiv:2506.13131.

[37] NVIDIA. Nemotron 3 Nano: Open, efficient mixture-of-experts hybrid Mamba-transformer model for agentic reasoning, 2025. arXiv preprint arXiv:2512.20848.

[38] NVIDIA, A. Blakeman, A. Grattafiori, et al. NVIDIA Nemotron 3: Efficient and open intelligence, 2025. arXiv preprint arXiv:2512.20856.

[39] NVIDIA, A. Chandiramani, et al. Nemotron 3 Super: Open, efficient mixture-of-experts hybrid Mamba-transformer model for agentic reasoning, 2026. arXiv preprint arXiv:2604.12374.

[40] K. Opsahl-Ong, M. J. Ryan, J. Purtell, et al. Optimizing instructions and demonstrations for multi-stage language model programs, 2024. arXiv preprint arXiv:2406.11695.

[41] C. Packer, S. Wooders, K. Lin, et al. MemGPT: Towards LLMs as operating systems, 2023. arXiv preprint arXiv:2310.08560.

[42] J. S. Park, J. C. O'Brien, C. J. Cai, et al. Generative agents: Interactive simulacra of human behavior, 2023. arXiv preprint arXiv:2304.03442.

[43] R. Pérez-Mohedano, N. Letzelter, C. Amador, C. T. VanderRoest, and S. Bakalis. Positron emission particle tracking (PEPT) for the analysis of water motion in a domestic dishwasher. *Chemical Engineering Journal*, 259:724–736, 2015.

[44] R. Pérez-Mohedano, N. Letzelter, and S. Bakalis. Integrated model for the prediction of cleaning profiles inside an automatic dishwasher. *Journal of Food Engineering*, 196:101–112, 2017.

[45] X. Puig, K. Ra, M. Boben, J. Li, T. Wang, S. Fidler, and A. Torralba. VirtualHome: Simulating household activities via programs. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8494–8502, 2018.

[46] C. Qian, W. Liu, H. Liu, et al. ChatDev: Communicative agents for software development, 2023. arXiv preprint arXiv:2307.07924.

[47] M. Robeyns, M. Szummer, and L. Aitchison. A self-improving coding agent, 2025. arXiv preprint arXiv:2504.15228.

[48] B. Romera-Paredes, M. Barekatain, A. Novikov, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

[49] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra. Habitat: A platform for embodied AI research. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9338–9346, 2019.

[50] N. Shinn, F. Cassano, E. Berman, et al. Reflexion: Language agents with verbal reinforcement learning, 2023. arXiv preprint arXiv:2303.11366.

[51] M. Shridhar, J. Thomason, D. Gordon, Y. Bisk, W. Han, R. Mottaghi, L. Zettlemoyer, and D. Fox. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10737–10746, 2020.

[52] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger. Defining and characterizing reward hacking, 2022. arXiv preprint arXiv:2209.13085.

[53] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. arXiv preprint arXiv:2408.03314.

[54] J. Snoek, H. Larochelle, and R. P. Adams. Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems 25 (NIPS 2012)*, 2012.

[55] S. Srivastava, C. Li, M. Lingelbach, R. Martín-Martín, F. Xia, K. Vainio, Z. Lian, C. Gokmen, S. Buch, C. K. Liu, S. Savarese, H. Gweon, J. Wu, and L. Fei-Fei. BEHAVIOR: Benchmark for everyday household activities in virtual, interactive, and ecological environments. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, pages 477–490, 2022.

[56] I. Voysey, T. George Thuruthel, and F. Iida. Autonomous dishwasher loading from cluttered trays using pre-trained deep neural networks. *Engineering Reports*, 3(5): e12321, 2021.

[57] A. Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186, 1945.

[58] F. Wang and K. Hauser. Stable bin packing of non-convex 3D objects with a robot manipulator. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8698–8704, 2019.

[59] G. Wang, Y. Xie, Y. Jiang, et al. Voyager: An open-ended embodied agent with large language models, 2023. arXiv preprint arXiv:2305.16291.

[60] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig. Agent workflow memory, 2024. arXiv preprint arXiv:2409.07429.

[61] Q. Wu, G. Bansal, J. Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023. arXiv preprint arXiv:2308.08155.

[62] C. Yang, X. Wang, Y. Lu, et al. Large language models as optimizers, 2023. arXiv preprint arXiv:2309.03409.

[63] X. Yin, X. Wang, L. Pan, et al. Gödel agent: A self-referential agent framework for recursive self-improvement, 2024. arXiv preprint arXiv:2410.04444.

[64] M. Yuksekgonul, F. Bianchi, J. Boen, et al. TextGrad: Automatic "differentiation" via text, 2024. arXiv preprint arXiv:2406.07496.

[65] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation, 2023. arXiv preprint arXiv:2310.02304.

[66] J. Zhang, J. Xiang, Z. Yu, et al. AFlow: Automating agentic workflow generation, 2024. arXiv preprint arXiv:2410.10762.

[67] J. Zhang, S. Hu, C. Lu, et al. Darwin gödel machine: Open-ended evolution of self-improving agents, 2025. arXiv preprint arXiv:2505.22954.

[68] A. Zhao, D. Huang, Q. Xu, et al. ExpeL: LLM agents are experiential learners, 2023. arXiv preprint arXiv:2308.10144.

[69] H. Zhao, Q. She, C. Zhu, Y. Yang, and K. Xu. Online 3D bin packing with constrained deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 741–749, 2021.

[70] L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena, 2023. arXiv preprint arXiv:2306.05685.

[71] M. Zhuge, W. Wang, L. Kirsch, et al. Language agents as optimizable graphs, 2024. arXiv preprint arXiv:2402.16823.